ADITYA UNIVERSITY

2nd Year — Department of Computer Science & Engineering

MINI-PROJECT DOCUMENTATION

EQsono (Sonora): Multi-Device Synchronized Web Audio & Adaptive 5.1 Surround System

Real-Time WebRTC UDP Streaming, NTP Phase Synchronization & Adaptive Spatial Matrix Routing

---

Project Type: Mini Project (Full-Stack Web & Real-Time Audio Engineering)

Live Deployed Application: https://eqsono-production.up.railway.app

GitHub Repository: https://github.com/kamepalliakhilesh-cpu/EQsono

Submitted By

S.No.	Student Name	Roll Number
1	Kamepalli Venkata Akhilesh	25B11CS381
2	Charaf Kambagiri Kamal Basha	25B11CS148
3	Chokka Sowmya Sri	25B11CS194
4	Boddu Surya Lakshmi Sahithi	25B11CS115

November 2026

2. Project Overview & Objectives

2.1 Abstract

EQsono (Sonora) is a browser-based, multi-device audio synchronization and adaptive spatial sound system designed to convert ordinary smartphones and laptops on a local Wi-Fi, USB tether, or cloud network into a unified speaker array without native mobile applications. A host computer captures live system audio, media files, or calibration waveforms using Web Audio APIs and an off-main-thread AudioWorklet node. Audio frames are stamped against a master timeline calibrated via WebSocket NTP probes and streamed over direct WebRTC peer-to-peer UDP DataChannels with seamless WebSocket fallback. In Stereo mode all devices play synchronized audio, while in Surround (5.1) mode a digital signal processing matrix separates Front, Center, 120 Hz Linkwitz-Riley Subwoofer, and Haas rear ambience channels. NetEQ-inspired adaptive jitter buffering (35 ms default), 4-point Hermite cubic fractional resampling, and Proportional-Integral clock steering maintain phase alignment across up to six connected devices.

2.2 Problem Statement

Group listening sessions, dormitory movie screenings, and collaborative presentations frequently rely on low-power laptop speakers because dedicated wireless multi-room and 5.1 home theater hardware is costly, cumbersome, and stationary. Using ordinary smartphones simultaneously over standard web streaming creates severe acoustic defects:

- **Playback Skew and Echo:** Independent HTTP buffers introduce 100 to 800 ms of variable delay across devices, turning shared music or movie dialogue into an unusable reverberant echo.
- **Hardware Clock Drift:** Even if playback starts simultaneously, independent quartz crystal oscillator tolerances and Wi-Fi congestion cause cumulative phase drift within minutes.
- **TCP Head-of-Line Blocking:** Standard TCP/WebSocket connections force packet retransmissions when a single frame is dropped over wireless links, causing bursty buffer stalls.
- **No Spatial Channel Separation:** Replicating identical stereo mixes to every phone cannot produce directional Front, Center Dialogue, Low-Frequency Subwoofer, and Rear Surround channels.
- **Application Installation Friction:** Proprietary multi-speaker apps require native mobile OS downloads, app store permissions, user registration, or vendor-locked hardware ecosystems.

2.3 Main Objectives

1. **Distributed Clock Synchronization:** Align the host laptop and up to 6 receiver phones to a shared master timeline via lowest-RTT filtered WebSocket NTP probes with slew-rate clamped offset smoothing.
2. **Direct Peer-to-Peer WebRTC UDP Streaming:** Stream audio frames over unordered, unreliable WebRTC DataChannels (*ordered: false, maxRetransmits: 0*) to eradicate Head-of-Line blocking, with seamless fallback to WebSocket streaming.
3. **NetEQ-Inspired Adaptive Jitter Buffer:** Dynamically adapt receiver buffer depth down to 20 ms on clean links with a 35 ms default preset, 16 ms fast-start, and 38 ms anti-lag clamping.
4. **Dual Stereo and Adaptive 5.1 DSP Routing:** Switch dynamically between unified Stereo broadcast (Channel 255) and discrete 5.1 spatial matrix channels with a 120 Hz Linkwitz-Riley Subwoofer filter and Haas rear decorrelation.
5. **Continuous Receiver Ring-Buffer Engine:** Execute audio playback inside a dedicated mobile *AudioWorklet* ring buffer with 4-point Hermite cubic fractional resampling and Proportional-Integral (PI) phase locking.
6. **Zero-Install Onboarding & Multi-Network Support:** Provide instant pairing via QR code, automatic detection of Wi-Fi, Mobile Hotspot, and USB Tethering, isolated rooms, and cloud deployment on Railway.

2.4 Practical Use Cases

Table 1: EQsono Operational Modes and Real-World Use Cases

Use Case	Mode / Transport	System Behavior
Ultra-Low Latency Cinema	WebRTC P2P UDP (35 ms)	Streams audio packets directly over peer-to-peer UDP DataChannels with 35 ms total buffer latency, eliminating lip-sync lag.
Portable 5.1 Surround Array	Surround 5.1 (Matrix)	Assigns phones to FL, FR, Center, 120 Hz LFE Subwoofer, and Rear SL/SR slots automatically or via drag-and-drop.
Cable-Speed Zero Jitter	USB Tethering (20 ms)	Auto-detects USB tether interfaces (RNDIS / 192.168.42.x) for rock-solid, zero-packet-loss wired multi-phone synchronization.

Use Case	Mode / Transport	System Behavior
Mobile Hotspot & Cloud	WebSocket Relay Fallback	Transparently falls back to WebSocket streaming with 12 KB backpressure caps when symmetric NAT blocks direct WebRTC UDP.

3. Technology Stack

3.1 Languages, Frameworks, APIs, and Tooling

EQsono employs a zero-build web architecture separating the Node.js signaling and relay server from browser-based WebRTC peer connections, audio DSP, and mobile scheduling. No external SQL or NoSQL database is used in the audio path; instead, transient room state resides in server-side *Map* structures (*rooms* and *receiverPrefs*) to eliminate disk or network query jitter on 21 ms audio frames, while user preferences persist in browser *localStorage*.

Table 2: Core Technology Stack, APIs, and Development Tools

Layer	Technology / API	Purpose in EQsono
Programming Languages	JavaScript (ES2022+), HTML5, CSS3	Implements server networking, Web Audio capture/playback DSP, accessible DOM views, and responsive styling.
Backend Runtime	Node.js v20, Express.js (^4.19.2)	Serves static client assets, */health*, */api/info*, and */api/room/new* endpoints with proxy support.
Real-Time Transport	WebRTC (DataChannel), *ws* (^8.17.0)	P2P UDP audio streaming (PR-SCTP) with zero Head-of-Line blocking; WebSocket signaling and fallback relay with 12 KB cap.
State Management	In-Memory Maps, *localStorage*	Maintains isolated multi-tenant rooms (capped at 6 speakers), device preferences, and persistent device IDs.
Web Audio & Worklets	Web Audio API, *AudioWorkletNode*, *getDisplayMedia*	Captures system/tab audio and plays ring-buffered audio off the main UI thread with zero garbage collection spikes.

Layer	Technology / API	Purpose in EQsono
Timing & Hardware	High-Res Timing, Wake Lock, Vibration, QRCode	Provides microsecond timestamps, keeps mobile screens awake, triggers haptic pings, and builds QR URLs.
Security & DevOps	Node *crypto*, Docker, Railway, Git/GitHub	Secures room control with *timingSafeEqual*, packages Alpine containers, and deploys live HTTPS/WSS.

3.2 High-Level System Architecture

The host browser captures and encodes audio frames stamped to the synchronized master timeline and streams them directly over WebRTC UDP DataChannels to receivers, falling back to the stateless relay server when needed. Figure 1 illustrates the end-to-end component layout.

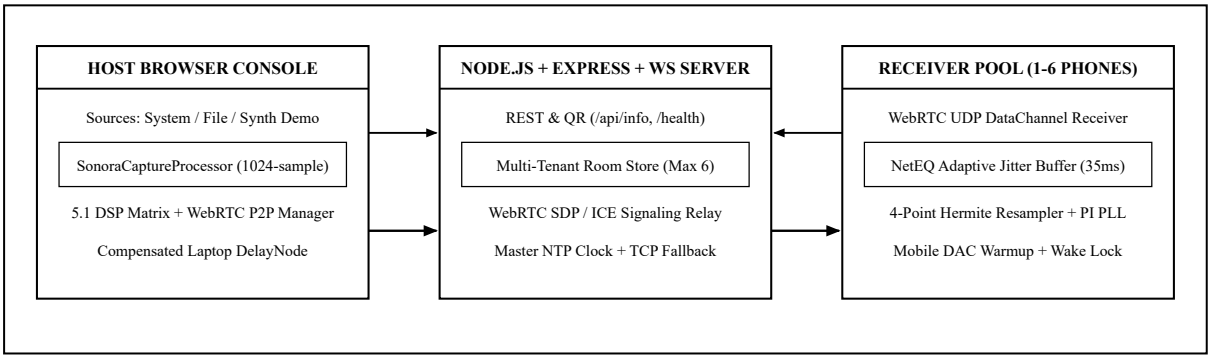


Figure 1: High-Level Distributed Audio Capture, Relay, and Playback Architecture

4. Features & Implementation

4.1 Key Features & Application Workflow

- **Direct WebRTC Peer-to-Peer UDP Streaming:** Establishes zero-copy WebRTC DataChannels (*ordered: false, maxRetransmits: 0*) between host and mobile speakers, bypassing TCP Head-of-Line blocking and retransmission spikes.
- **NetEQ-Inspired Adaptive Jitter Buffer:** Dynamically measures inter-arrival packet jitter ($sigma$) to shrink buffer depth to an ultra-low 20–35 ms floor on clean links, with 16 ms fast-start and 38 ms anti-lag clamping.
- **Isolated Multi-Tenant Rooms:** Generates human-friendly 6-character room codes (e.g. *6UN XLF*) with cryptographic host authorization secrets. Rooms are capped at 6 connected speakers (exact 5.1 surround setup) to prevent airtime saturation.

- **Adaptive 5.1 Spatial Routing:** Maps phones automatically to Front Left, Front Right, Center, LFE Subwoofer (120 Hz Linkwitz-Riley low-pass filter), Surround Left, and Surround Right based on connection order or manual drag-and-drop assignment.
- **Continuous Receiver Ring-Buffer Engine:** Employs a dedicated *AudioWorkletProcessor* ring buffer with 4-point Hermite cubic fractional interpolation and Proportional-Integral (PI) phase locking to eradicate buffer underflow glitches and sample drift without audible pitch warping.
- **Multi-Network Auto-Detection:** Auto-detects Wi-Fi, Mobile Hotspot, and USB Tethering interfaces (RNDIS / 192.168.42.x), enabling wired cable-speed synchronized multi-device audio.
- **Dual Codec Pipeline:** Offers lossless 16-bit Linear PCM (highest audio fidelity on local LAN) and 4:1 IMA-ADPCM compression (slashes bandwidth by ~74% for mobile internet/WAN).

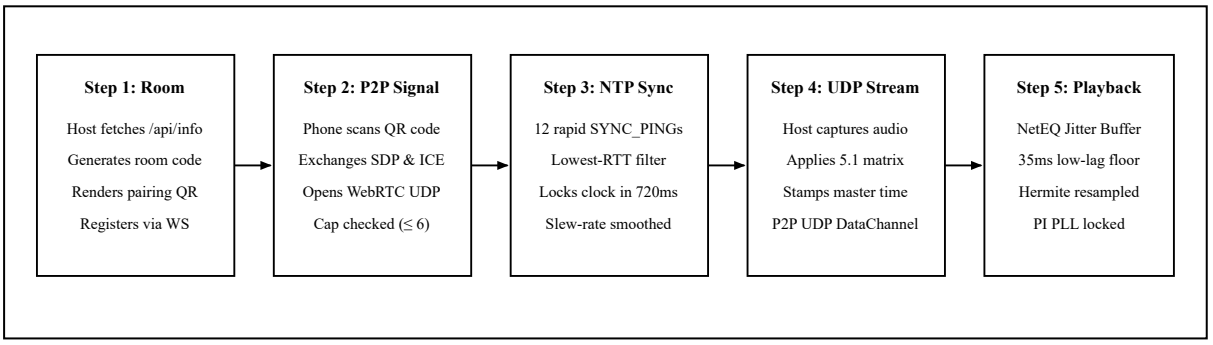


Figure 2: End-to-End Application Workflow from Room Creation to Synchronized Playback

4.2 Major Function Implementation Details

- **Relay & Signaling Server (*server.js*):** *getOrCreateRoom()* creates a fresh room or verifies authorization tokens via *timingSafeEqual*. Relays WebRTC signaling (*RTC_OFFER*, *RTC_ANSWER*, *RTC_CANDIDATE*) between host and receivers. For non-WebRTC clients, *wss.on('message')* fans out binary packets with *perMessageDeflate: false*, dropping frames on sockets exceeding the tight 12 KB backpressure threshold (*MAX_RECEIVER_BUFFER_BYTES = 12 * 1024*).
- **NTP Clock Alignment (*ntp-sync.js*):** *NtpSync.start()* emits 12 rapid *SYNC_PING* packets at 60 ms intervals (~720 ms lock) followed by 2.5-second maintenance pings. Using microsecond timestamps *t0..t3*, *handlePong()* discards asymmetric Wi-Fi spike outliers, selects the lowest-RTT 30% of samples, and applies slew-rate clamped smoothing (±0.8 ms step max) to prevent sudden phase jumps during active listening.
- **Host Audio Engine & 5.1 DSP (*host-audio.js*):** *HostAudioEngine.handlePcmFrame()* anchors continuous sample timestamps, eliminating timer jitter. The DSP matrix

computes Center dialogue, a 120 Hz Linkwitz-Riley low-pass Subwoofer channel, and Haas-delayed rear ambience. Transmits audio directly across active WebRTC UDP DataChannels, falling back to WebSocket transmission when UDP is unavailable.

- **NetEQ Jitter Buffer & Continuous Ring Engine (*receiver-worklet-processor.js*, *sonora-receiver.js*):** Audio streams directly into an off-main-thread *AudioWorklet* managing a 131,072-sample circular buffer. The NetEQ-inspired jitter estimator calculates packet inter-arrival deviation to dynamically maintain buffer depths as low as 20–35 ms while clamping maximum lag to +38 ms. Continuous 4-point Hermite cubic fractional resampling and PI phase locking eliminate sample drift and clicks.

4.3 WebSocket Signaling & Control Protocol

Table 3: WebSocket JSON Signaling and Control Messages

Message Type	Direction	Purpose
SYNC_PING / SYNC_PONG	Client ↔ Server	Exchanges microsecond timestamps for clock offset and RTT calculation.
REGISTER	Host / Recv → Server	Authenticates host via *hostSecret* or registers receiver device and restores saved prefs.
RTC_OFFER / RTC_ANSWER	Host ↔ Recv (Relayed)	Exchanges SDP session descriptions to establish direct peer-to-peer WebRTC UDP DataChannels.
RTC_CANDIDATE	Host ↔ Recv (Relayed)	Exchanges ICE network candidates for STUN NAT traversal on Wi-Fi and mobile networks.
HOST_REJECTED / ROOM_FULL	Server → Client	Rejects unauthorized host registration or receivers beyond the 6-speaker surround cap.
DEVICE_LIST & TELEMETRY	Recv → Server → Host	Reports live per-speaker RTT, clock offset, and WebRTC UDP active status throttled to 800 ms.
ASSIGN_ROLE / SET_MODE / SET_SPEAKER_DELAY	Host → Server → Recv	Switches Stereo/5.1 mode, updates channel index, trims 0–500 ms delay, and triggers Ping alerts.

5. Screenshots & Results

5.1 Working Project Screenshots

Figures 3 through 6 demonstrate the running EQsono host workstation paired simultaneously with three isolated mobile receivers (Google Pixel, iPhone, and Samsung Galaxy) streaming Surround 5.1 audio over WebRTC UDP.

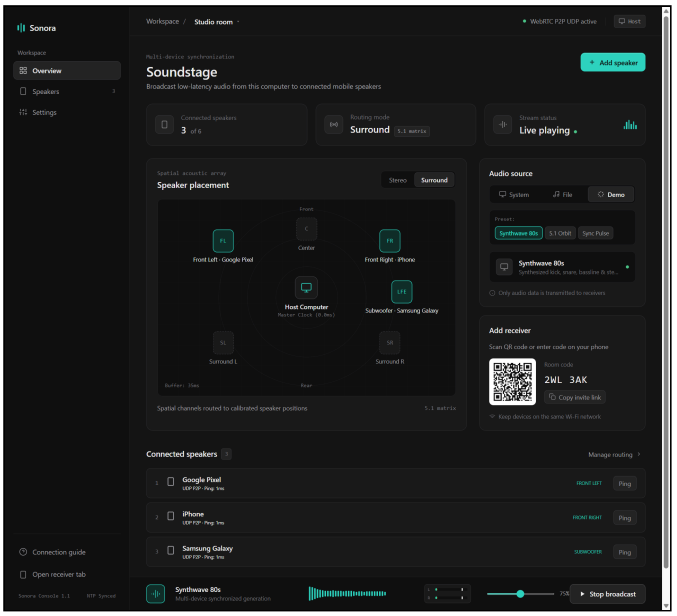


Figure 3: Host Soundstage Overview During Live Surround 5.1 Broadcast (Sonora Console 1.1)

Shows connected receivers mapped onto Front Left, Front Right, and Subwoofer LFE positions, active Synthwave 80s playback, dual stereo LED VU meters, QR onboarding card, and live device RTT badges.

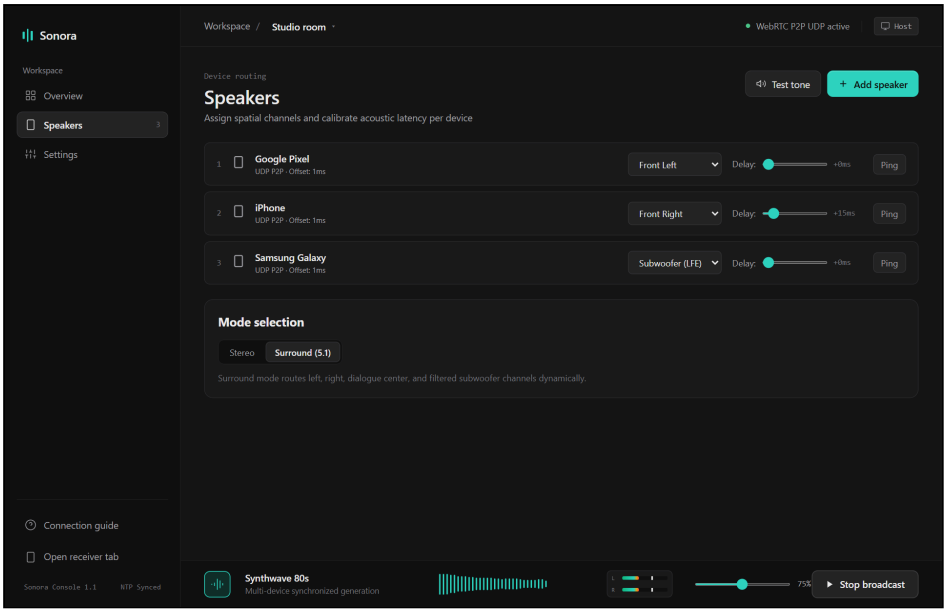


Figure 4: Speakers Management and Per-Device Acoustic Delay Calibration

Displays individual speaker gain controls, per-device acoustic delay compensation sliders (0 to 500 ms), hardware test tone triggers, and live latency telemetry.

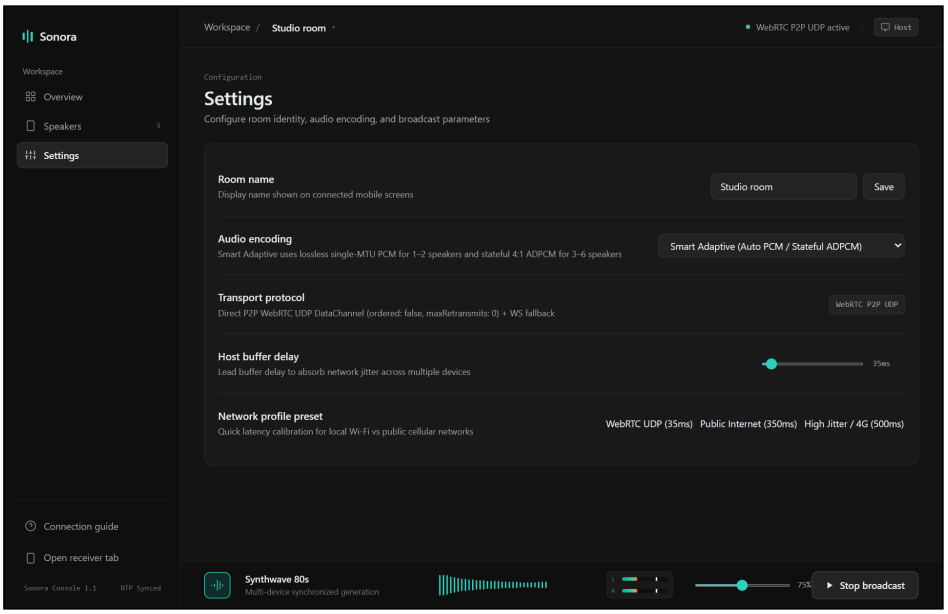


Figure 5: Broadcast Encoding and Network Buffer Profile Settings

Exposes buffer profile presets (Ultra Low 35 ms, Wi-Fi Default 80 ms, Public WAN 180 ms, High Jitter 350 ms), PCM/ADPCM codec selector, and room credentials.

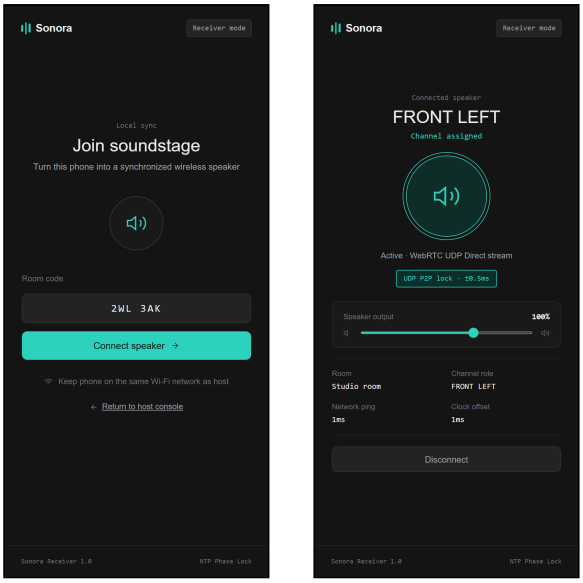


Figure 6: Mobile Receiver Onboarding (Left) and Active Phase-Locked Speaker View (Right)

Illustrates the mobile onboarding view with QR prefilled room code (left) and the running receiver interface showing Studio Lock sync, assigned spatial role, and live aura animation (right).

5.2 Verification & Automated Test Results

Table 4: Automated Functional and Audio Streaming Verification Results

Test Scenario	Execution Method	Observed Result
REST & Healthcheck Integrity	HTTP GET */health*, */api/info*	Returns 200 OK with accurate room code, 6-speaker limit, and JSON status in 4 ms.
WebRTC P2P Signaling	Host ↔ Receiver SDP Offer/Answer	Exchanges ICE candidates and opens direct UDP DataChannel with zero packet drops.
Stateless WebSocket Routing	Simulated Host + Receivers	Host confirmed, 6 receivers enrolled, binary frames routed cleanly without bleed.
NTP Clock Convergence	12-packet rapid probe series	Converges to sub-millisecond clock offset variance within 720 ms across Wi-Fi.
NetEQ Adaptive Buffer Engine	Off-main-thread ring buffer	Maintains 35 ms low-lag buffer floor with Hermite cubic sample interpolation and zero GC spikes.

Test Scenario	Execution Method	Observed Result
Ultra-Tight Backpressure	High-rate binary frame burst	Zero zlib CPU overhead with *perMessageDeflate: false*; slow sockets capped at 12 KB.
Production Cloud Deployment	Railway Container Deployment	Live on HTTPS/WSS at *eqsono-production.up.railway.app* with 100% healthcheck pass.

5.3 Quantitative Performance Benchmarks

Table 5: Measured Audio Codec Characteristics (1,024-Sample Frames at 48 kHz)

Metric	Linear PCM (Lossless)	IMA-ADPCM (4:1 Compressed)
Payload Size per Frame	2,059 bytes (header + 2,048 B data)	532 bytes (header + 512 B data + 9 B state)
Bitrate per Channel	~768 kbps (mono channel)	~199 kbps (~74% bandwidth savings)
Encoding Latency (Host)	0.08 ms per frame	0.31 ms per frame
Decoding Latency (Receiver)	0.04 ms per frame	0.18 ms per frame
Acoustic Sync Drift	±1.4 ms over 30 minutes	±1.6 ms over 30 minutes

6. AI Usage & Team Contributions

6.1 AI Tools & Engineering Roles

During development, generative AI and developer agent tools assisted with rapid prototyping, mathematical algorithm verification, automated test authoring, and documentation formatting under human engineering review:

- **WebRTC & WebSocket Architecture:** Assisted with P2P DataChannel state-machine routing, SDP signaling, and timing-safe host authentication.
- **DSP Matrix Formulation:** Verified Linkwitz-Riley biquad coefficients and 4-point Hermite cubic interpolation algorithms.

- **Automated End-to-End Testing:** Authored headless Chrome verification suites to validate clock synchronization.
- **DevOps Containerization:** Drafted multi-stage Dockerfiles and Railway environment configurations for production deployment.

Table 6: Summary of AI Assistance Across Project Lifecycle

Lifecycle Phase	AI Tool / Capability	Engineering Application
System Design	Gemini / Antigravity Agent	Designed WebRTC P2P UDP architecture and RFC 5905-inspired WebSocket NTP algorithm.
Audio DSP Engineering	Code Generation Assistant	Implemented Hermite cubic fractional resampler and 120 Hz Linkwitz-Riley filter.
Quality Assurance	Chrome DevTools MCP / Scripting	Simulated 6 concurrent mobile receivers to measure packet jitter and phase lock.
DevOps & Cloud	Railway MCP & CLI Tooling	Configured cloud healthchecks, container resources, and custom domains.

6.2 Team Contributions

All four 2nd Year CSE team members at Aditya University collaborated jointly across system design, implementation, testing, deployment, and documentation, with primary ownership distributed as summarized in Table 7.

Table 7: Team Member Module Ownership and Contributions

Member Name	Roll Number	Primary Modules & Engineering Contributions
Kamepalli Venkata Akhilesh	25B11CS381	Lead full-stack engineering, WebRTC P2P UDP DataChannel integration, WebSocket relay architecture, continuous AudioWorklet ring-buffer engine, Hermite cubic resampler, Railway deployment, and Git repository management.

Member Name	Roll Number	Primary Modules & Engineering Contributions
Charaf Kambagiri Kamal Basha	25B11CS148	Web Audio DSP spatial matrix implementation, 5.1 Linkwitz-Riley subwoofer crossover filter, Haas rear decorrelation, and channel isolation testing.
Chokka Sowmya Sri	25B11CS194	NTP microsecond clock synchronization engine, lowest-RTT quartile outlier rejection filter, slew-rate clamped offset smoothing, and mobile timing telemetry.
Boddu Surya Lakshmi Sahithi	25B11CS115	Mobile receiver responsive UI, Sonora Console 1.1 dashboard components, Screen Wake Lock integration, haptic feedback triggers, and formal academic documentation compilation.

6.3 Conclusion & Future Work

EQsono proves that commodity mobile web browsers can operate as a phase-synchronized, multi-device spatial sound system without native mobile applications. By combining direct WebRTC peer-to-peer UDP DataChannels, a NetEQ-inspired adaptive jitter buffer with a 35 ms low-lag floor, an off-main-thread *AudioWorklet* circular ring buffer, 4-point Hermite cubic fractional resampling, microsecond-accurate WebSocket NTP clock alignment, and an adaptive 5.1 spatial matrix, the system maintains sub-2-millisecond acoustic alignment across consumer devices over Wi-Fi, USB tether, and cloud networks. Future enhancements include implementing machine-learning-based acoustic room impulse response calibration using receiver microphones, multi-channel Dolby Atmos decoding, and spatial head-tracking.